

Rapport d'implémentation

MUZEAU Raphaël contact: raphael.muzeau@etudiant.univ-perp.fr DANNA-VASSEUR
Lucas contact: lucas.danna--vasse@etudiant.univ-perp.fr CORTALE-BECKER Tom
contact: tom.cortale-beck@etudiant.univ-perp.fr

Architecture de l'arborescence

La repo est organisée de la manière suivante :

```
├── bin
├── build
├── data
│   ├── ...
│   ├── parties
│   ├── pictures
│   └── test
├── lib
├── rapport
├── src
│   ├── controller
│   ├── model
│   └── vue
```

La seule différence notable avec la partie analyse du rapport et que les artefacts de compilations ont migrés vers `build/`, afin de souligner que ce dossier n'est plus limité au stockages des objets `.o`, mais contient également des sous-makefiles `.d` générés par gcc pour traquer les dépendances entre les fichiers sources et headers.

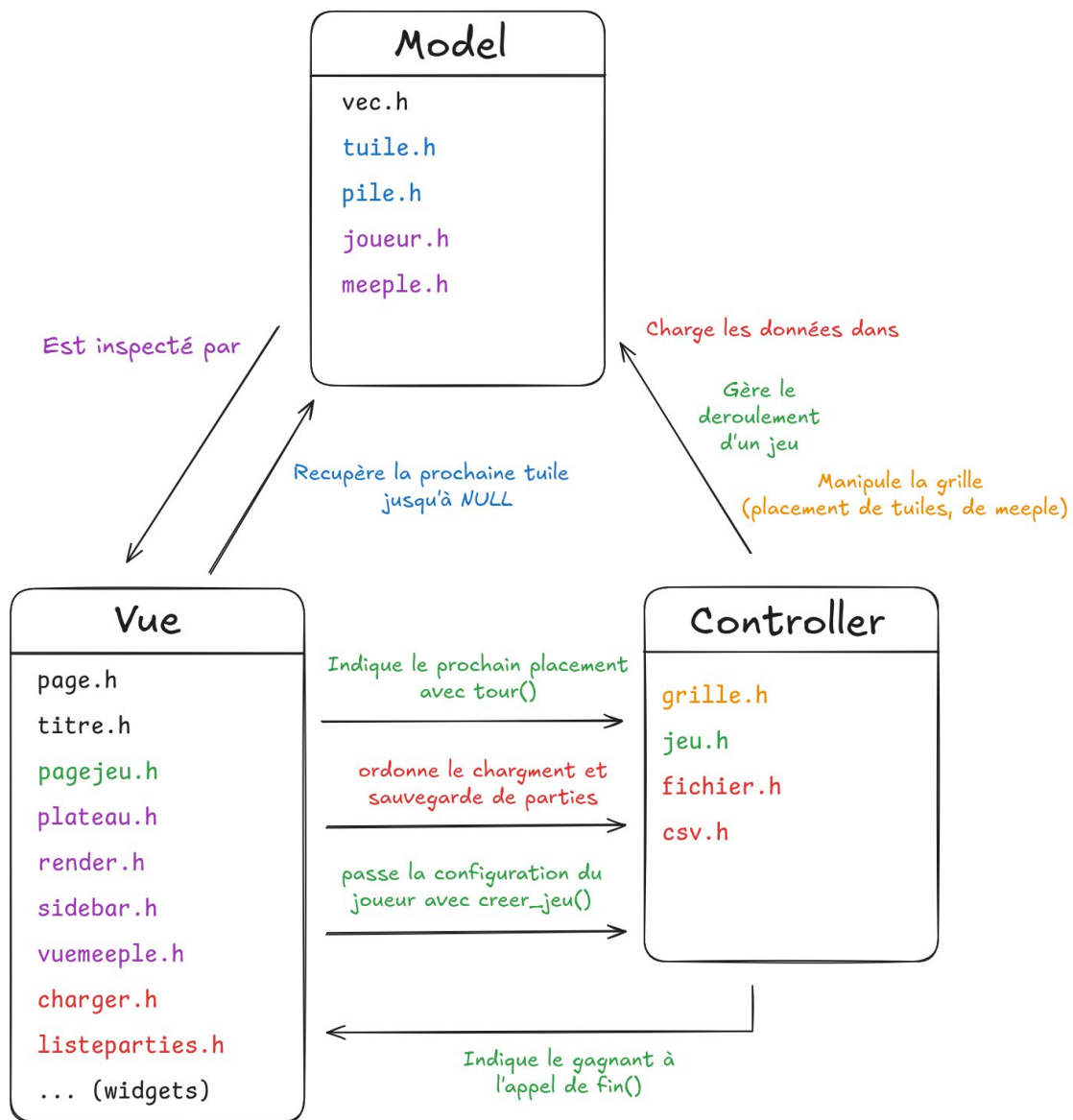
Même si le fonctionnement détaillé du makefile ne sera pas décrit ici, quelques points sont à souligner :

Trois cibles de compilations différentes sont disponibles : - `release`, voué à la distribution finale du programme, compilé avec optimisations - `debug`, voué à être inspecté avec des outils de debug durant le développement - `test`, un mode spécial de debug qui lance la suite de test sous l'outil `valgrind` s'il est disponible

De plus, nous avons décidé de faire usage d'un LSP (`clangd`) pour fluidifier le processus écriture -> compilation -> test, son usage est optionnel, mais le makefile permet de générer le fichier `compile_commands.json` nécessaire à son fonctionnement avec la commande `make bear`.

Model MVC

Les structures de données et fonctions utilisés seront décrites plus bas, en premier lieu, voici un schema explicatif des interactions entre le modèle, la vue et le controller:



Librairie de support

Nous avons élaboré deux petites librairies pour soutenir le reste de la base de code.

test

Le header test.h propose la fonction `_Noreturn void lancer_tests(void)` appelée par main à la place du corps si le fichier est compilé avec `RUN_UNIT_TESTS` définie (par `make test`).

Cette fonction executera toutes les fonctions de test définies dans test.c, et génère un rapport sur leurs resultats de la forme suivante:

lancement de la suite de tests:

```

test_tuile_creer: ok
test_tuile_pivot90: ok
test_tuile_compatibilite: ok
test_tuile_generer: ok
test_pile_creer: ok
test_pile_creer_aleatoire: ok
  
```

```

test_pile_inserer_tuile: ok
test_pile_recuperer_tuile_non_aleatoire: ok
test_pile_recuperer_tuile_aleatoire: ok
test_vec_creer: ok
test_vec_get_null: ok
... # omis pour ne pas polluer ce document
test_csv_compter_lignes: ok
test_csv_lecture_zone: ok
test_csv_lecture_fichier: ok
test_csv_fichier_vide: ok
test_csv_fichier_introuvable: ok
test_csv_fichier_invalide: ok
test_fichier_charger_grille: ok
test_fichier_charger_pile: ok
test_fichier_charger_pile_aleatoire: ok
test_fichier_charger_joueur: ok
test_fichier_charger_joueur_liste_vide: ok
test_fichier_sauvegarder_liste_joueurs: ok
resultats:
    50 ok, 0 echecs.. exiting

```

Ces fonctions sont vouées à tester l'intégralité du modèle et du controller, généralement avec un ou plusieurs tests par fonction "publique"; elle sont de la forme suivante :

```
bool test_nomdefichier_nomdefonction(void)
```

Leur pointeurs et leur noms sont ensuite tous ajoutés à la liste statique `unit_tests[]` dans l'ordre d'exécution voulu : (un test qui utilise un fonction "foo" doit être lancé après le test de la fonction foo.). Qui sera lue par `lancer_tests`.

Le pointeur de fonction et le nom de chaque fonction sont groupés à l'aide de la structure :

```

typedef struct _Test {
    bool (*run)(void);
    const char *name;
} Test;
#define TEST(FUNCTION) { FUNCTION, #FUNCTION }

```

Notons la macro `TEST`, utilisé pour faciliter l'ajout d'un test à `unit_tests[]` comme tel :

```

// ajout à la liste de tests à executer
Test unit_tests[] = {
    ...
    TEST(test_nomdefichier_nomdefonction),
    ...
};

```

libca

`libca` (lire: lib carcassonne) offre deux types de fonctions:

- `void *ca_alloc(size_t n, size_t size);`
- `void *ca_realloc(void *p, size_t n, size_t size);`
- `void ca_warn(const char *message);`
- `_Noreturn void ca_error(const char *message);`
- `_Noreturn void ca_perror(void);`

`ca_alloc` et `ca_realloc` sont des wrappers autour de `calloc` et `realloc` respectivement, avec deux sécurité ajoutées: déclenche un crash propre avec message d'erreur lors d'une erreur d'allocation, ou si la multiplication de `n` et `size` déclenche un overflow.

Les fonctions `ca_warn`, `ca_error`, `ca_perror` affichent une erreur avec un format standardisé, et crash proprement en cas d'erreur.

Ainsi qu'une macro `max(A, B)` pour simplifier cette operation.

Structures de données

Cette section sera découpé en sous-chapitres qui représentent chaque structure de données. Chaque structure présentera aussi les fonctions importantes à souligner utilisées pour interagir avec elle. Les fonctions plus simples sont omises par soucis de lisibilité, et car elles portent plutôt un rôle de soutien à ces fonctions primaires.

Tuiles :

Le premier élément sur laquelle nous allons nous concentrer est la tuile, pouvant être retrouvés dans `tuile.h` et ses fonctions dans `tuile.c`.

La première chose à laquelle nous avons dû penser est la représentation des zones et leurs valeurs. Il a donc été décidé, après quelques réflexions, que la structure la plus correcte serait la suivante :

```
enum Zone {
    Z_PRE      = 0x01, // 0b000001
    Z_ROUTE    = 0x02, // 0b000010
    Z_VILLE    = 0x04, // 0b000100
    Z_BLASON   = 0x0c, // 0b001100
    Z_VILLAGE  = 0x10, // 0b010000
    Z_ABBAYE   = 0x20, // 0b100000
};
```

Il est à noter qu'il y a un bit en commun entre `Z_VILLE` et `Z_BLASON` pour que les deux puissent être considérées égales via l'opération `&` ("et binaire") en C.

Vient ensuite l'implémentation de la tuile, elle-même. Nous avons donc eu à réfléchir aux informations dont nous avons besoin dans une tuile :

- Nos zones (milieu, nord, sud, est, ouest),
- Un id meeples pour savoir si un meeples est posé sur la tuile,
- La position du meeples,
- Savoir si on a vérifié notre tuile lors de la recherche,

```
struct _Tuile {
    enum Zone milieu, nord, sud, est, ouest;
```

```

    int id_meeple;
    enum Direction position_meeple;
    bool est_verifie;
};
typedef struct _Tuile *Tuile;

```

Il est à noter que le type Tuile est un **pointeur**. La variable id_meeple aura une valeur supérieure à 0 si un meeple est posé dessus, -1 s'il n'y a aucun meeple sur la tuile. Il est importante de re-préciser qu'il ne peut y avoir qu'**un seul meeple** par tuile. Le booléen est_verifie variera entre false et true durant la recherche, il indique si la tuile a déjà été évaluée par l'algorithme durant ce passage. Il nous faut aussi un moyen de connaître la position de notre meeple, nous avons donc créer une dernière structure utile au repérage du meeple sur la tuile :

```

enum Direction {
    D_SUD = 0,
    D_NORD,
    D_EST,
    D_OUEST,
    D_MILIEU,
};

```

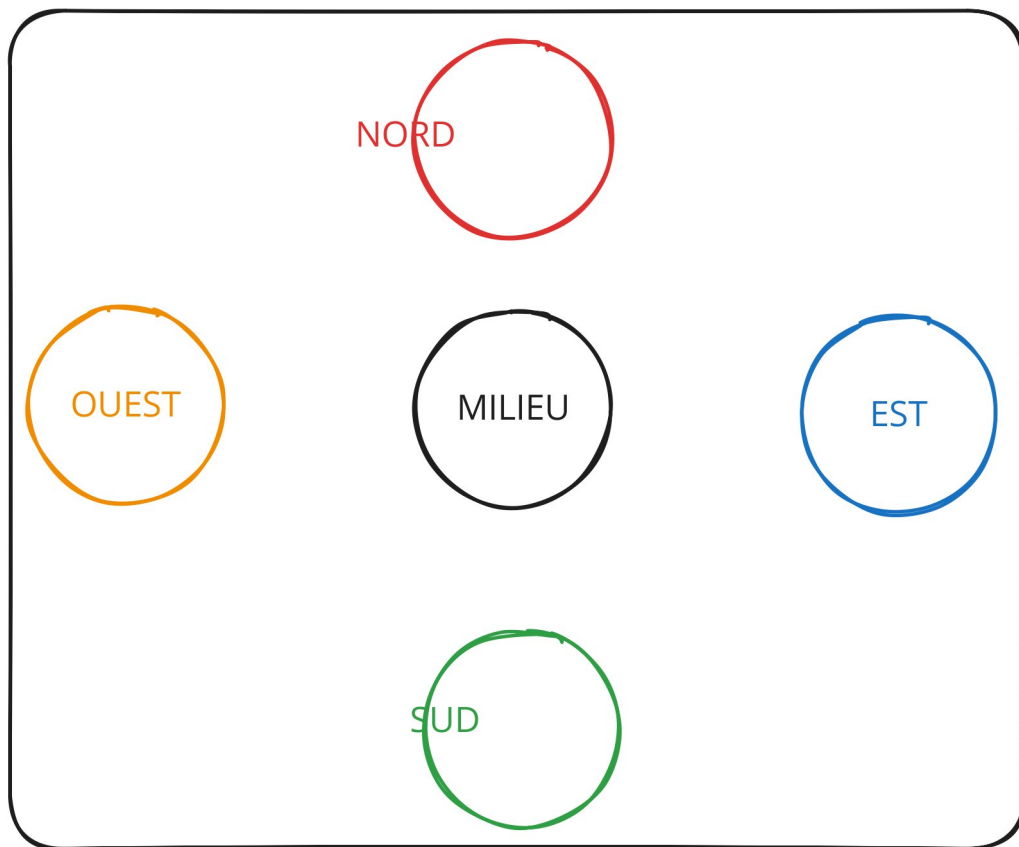
Cette structure aura aussi d'autres utilités dans la *recherche*.

Notre tuile ainsi définie interagit avec différentes fonctions, en voici une liste exhaustive (via leurs signatures) qui peut être retrouvée dans tuile.h :

- *Tuile creer_tuile(void)*
- *void pivot_90(Tuile piece)*
- *enum Zone zone_tuile(Tuile t, enum Direction d)*
- *bool compatibilite_tuile(Tuile depart, Tuile arrivee, enum Direction d)*
- *Tuile generer_tuile(void)*

Schéma de la tuile :

:



Précisions sur certaines fonctions :

La fonction **creer_tuile** renvoie une tuile initialisée comme un pre (chaque zone à la valeur d'un pré), l'id_meeple sera à -1, la direction du meep1e sera 0 (D_SUD) et le booléen à false (0);

La fonction **pivot_90**, réalise le pivot sur la droite.

La fonction **zone_tuile** nous renvoie la valeur d'une zone de la tuile selon une direction donnée.

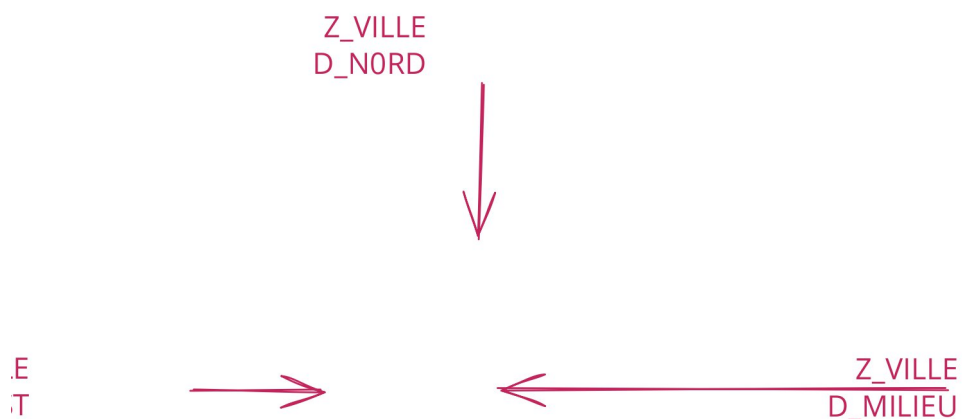
La fonction **compatibilite_tuile** nous renvoie un booléen qui nous dit si deux tuiles sont compatibles entre elles.

La fonction **generer_tuile** sert à générer une tuile (respectant les règles du jeu) de façon aléatoire.

Générer tuiles :

La génération aléatoire est une des demandes à respecter ajoutée à l'analyse du projet. Pour pouvoir la réaliser nous avons étudié longuement la construction des tuiles du .csv et avons ajouté quelques règles pour créer des tuiles dites "valides" dans notre version de Carcassonne. Une règle implicite que nous ajoutons au jeu est le fait que si

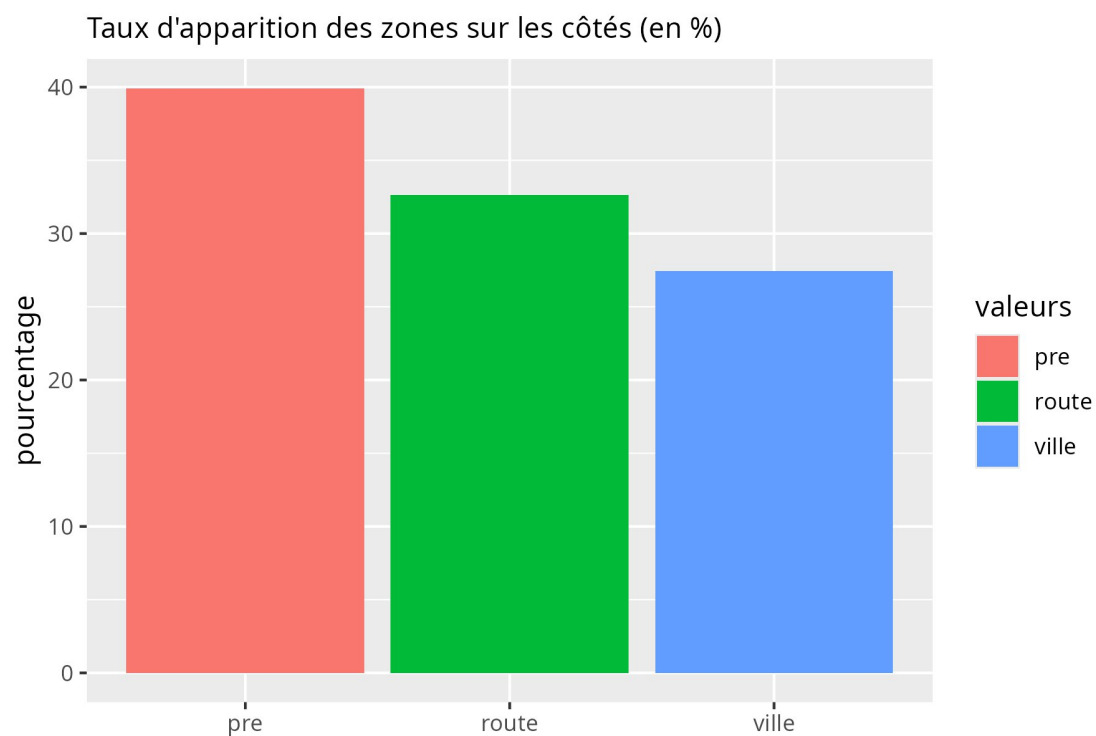
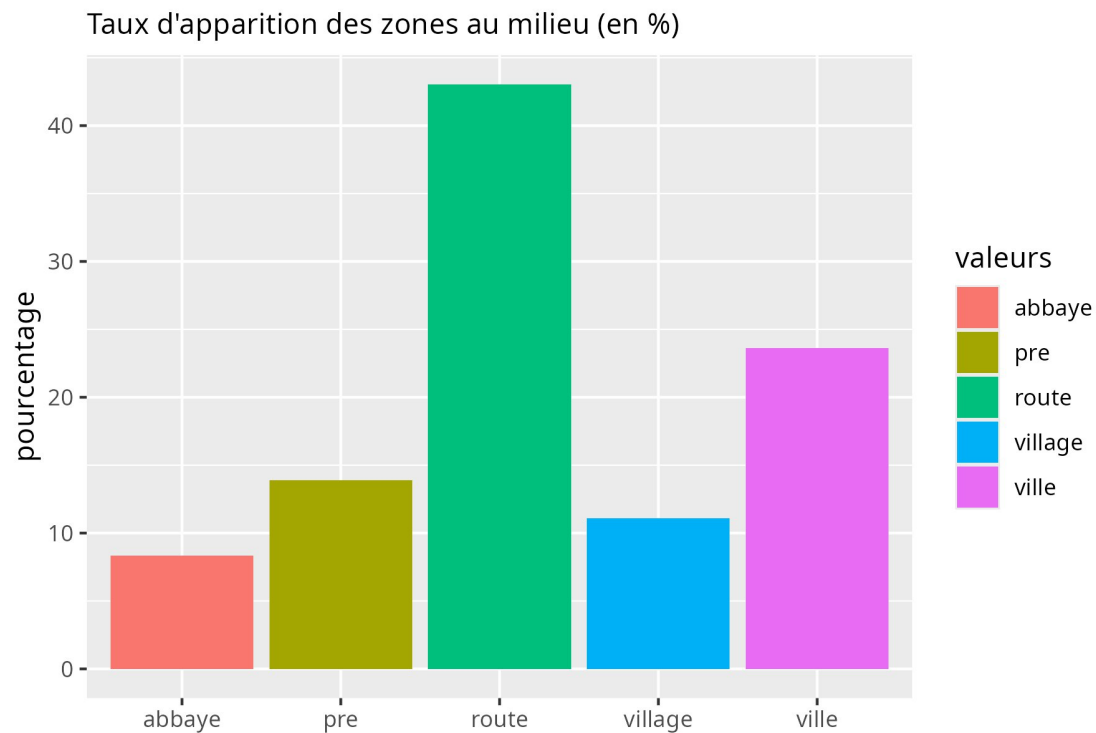
une zone “route” ou “ville” apparaît au milieu, alors celle-ci fait le lien entre deux autres zones, comme sous cet exemple :



Nous autorisons aussi la génération de tuiles qui n’existent pas dans le Carcassonne d’origine, comme celles-ci, par exemple :

Pour encore plus de fun !

Les probabilités pour la génération des tuiles ont été calculées via un script R se basant sur le .csv fourni pour la réalisation du projet, en voici les diagrammes en barres :



Meeple :

Le meeples n'est pas une structure en elle-même dans notre implémentation. Nous avons réfléchi à son utilisation et nous nous sommes rendus compte que le meeples n'avait pas besoin d'une structure en dédiée. Par contre, nous devons avoir un endroit qui nous sert à connaître en permanence les positions des meeples.

Nous avons donc eu l'idée d'une liste chaînée de meeples, qui nous sert à n'importe quel moment à savoir où se trouvent les meeples.

Voici la structure implémentée :


```

struct _Maillon {
    struct _Maillon *next;
    int x;
    int y;
};
typedef struct _Maillon *L_meeple;

```

S'accompagnant de toutes les fonctions classiques d'une liste chaînée :

- *L_meeple creer_maillon_meeple(int x, int y, enum Direction d)*
- *void detruire_liste_meeple(L_meeple liste)*
- *void ajouter_maillon_meeple(L_meeple *liste, L_meeple nouveau)*
- *void retirer_maillon_meeple(L_meeple *liste, int x, int y)*

Pile :

Il nous fallait aussi implémenter une pile de tuiles, dans laquelle nous pourrions ranger les tuiles d'un .csv ou même compter les tuiles restant aux joueurs. En respectant les deux implémentations possibles donnée dans l'analyse. La structure finale nous a donnée ceci :

```

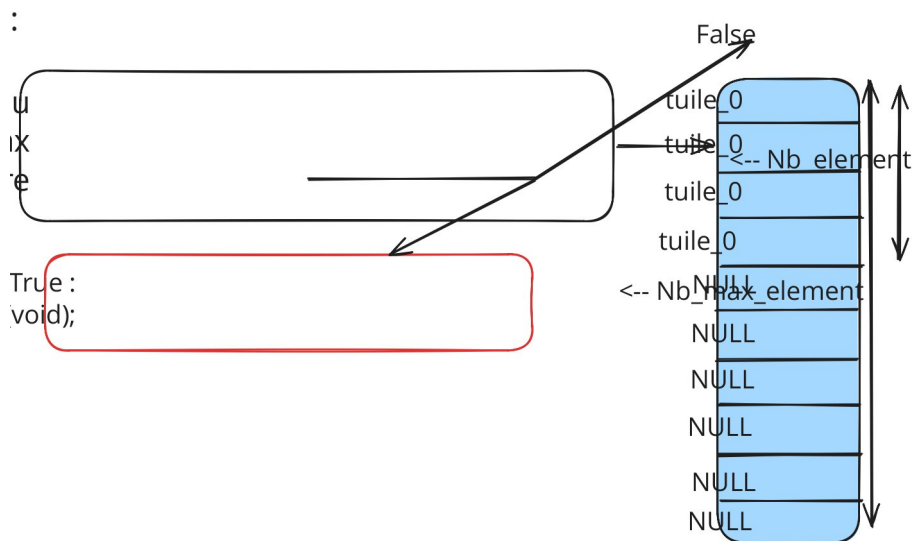
typedef struct _Pile {
    Tuile *tab;
    bool gen_aleatoire;
    int nb_element, nb_element_max;
} Pile;

```

Avec les fonctions suivantes :

- *Pile creer_pile(int max_element, bool gen)*
- *bool pile_vide(Pile p)*
- *bool pile_pleine(Pile p)*
- *Tuile recup_tuile(Pile *p)*
- *bool inserer_tuile(Pile *p, Tuile t)*
- *void detruire_pile(Pile *p)*

Schéma de la pile :



Précisions sur certaines fonctions :

La fonction **recup_tuile** quand *gen_aleatoire* vaut false, récupère une tuile au hasard comprise entre [0, nb_element-1]. Une fois la tuile récupérée, son emplacement est de nouveau remplie avec la dernière tuile se trouvant dans la pile. Si *gen_aleatoire* vaut false, **recup_tuile** appellera la fonction **generer_tuile**.

La fonction **bool inserer_tuile** est une fonction qui nous servira seulement lors de la lecture de .csv. Elle insère une Tuile dans le tableau de la pile.

Plateau :

La question de la représentation du tableau était restée longtemps un sujet d'étude. Plusieurs suggestions ont été proposées et ont vu le jour, comme :

- Tableau 2D statique : Avantages : simple, pratique, très rapide à traverser. Désavantages : Coûteux en mémoire, nécessite $(2*n)^2 * \text{sizeof}(\text{Tuile})$ octets pour n tuiles.
- Listes n-chaînées : Avantage : Réduction du coup mémoire Désavantage : Complexité d'utilisation liée à la recherche, cout en temps des dereferencement, enchaînements implicites (localisation compliqué lors de l'enchaînement)

Nous étions à l'origine parti sur un simple tableau 2D statique, dont la taille restait personnalisable. Mais lors de l'implémentation de la structure de Varstring, une idée a émergée. Celle du **Vec2D**, un tableau sans allocation mémoire initiale, indexable de INT_MIN à INT_MAX, donc même dans les negatifs. Ceci a plusieurs avantages évidents pour nous:

- On conserve la rapidité d'iteration d'un tableau statique.
- Un nombre de tuiles maximal important n'implique pas une allocation excessive.
- Fait coïncider l'index dans la grille et l'index du sprite dans la vue.
- Ne nécessite aucune verification de bord durant la recherche.

Le coût mémoire était grandement réduit, et nous gardions la simplicité d'accessibilité d'un tableau via des fonctions très simple d'accès.

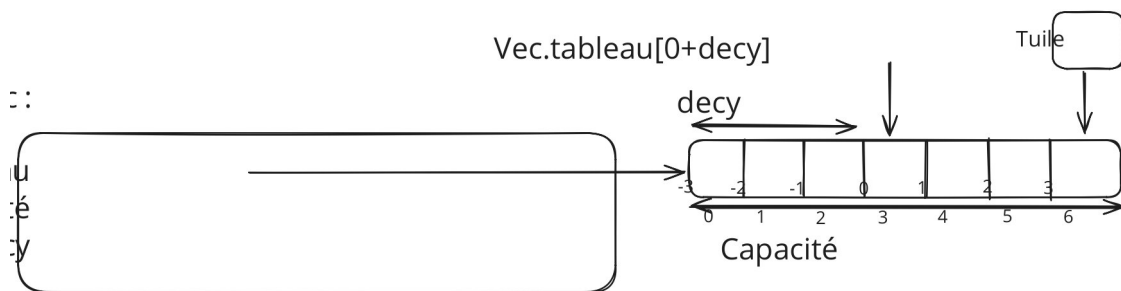
Le Vec2D repose sur deux structures presque identiques. La première :

```
typedef struct _Vec {  
    Tuile *_tableau;  
    int _capacite; // nombre de tuiles pouvant être contenues  
    int _decy;  
} Vec;
```

Cette structure Vec est celle qui contiendra les Tuiles, la variable *decy* nous est utile pour pouvoir indexer avec des valeurs négatives. Si une tuile est posée en -1 sur le plateau, le décalage augmente de 1, pour que lors de l'indexation sur *_tableau, la valeur soit tout de même comprise entre [0, _capacite].

Un vec est manipulé de manière analogue à celle d'un tableau standard, grâce à aux fonctions de placement et d'accès `vget(Vec, int x, int y)` et `vset(Vec*, Tuile, int x, int y)`. Toutes les cases sont, du point de vue de l'utilisateur, initialisées à NULL.

Voici un schéma visuel de ce que fait la structure :

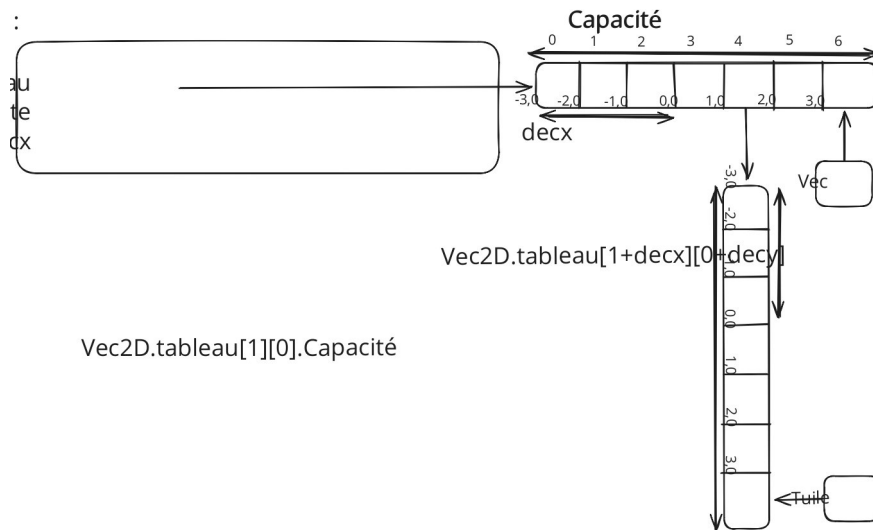


Cette structure sert donc à gérer les *colonnes* de notre plateau, ne reste plus qu'à gérer les *lignes*. C'est là qu'intervient notre 2nd structure :

```
typedef struct _Vec2D {  
    Vec *_tableau;  
    int _capacite;  
    int _decx;  
} Vec2D;
```

Le principe y est le même, notre tableau indexe ici des *Vec*, mais tout le reste fonctionne de la même manière.

Accompagné de son schéma :



Liste exhaustive des fonctions :

- ***Vec2D creer_vec2D(void)***
- ***void detruire_vec2D(Vec2D g)***
- ***Tuile get(Vec2D g, int x, int y)***
- ***void set(Vec2D *g, Tuile t, int x, int y)***

Précisions sur certaines fonctions :

La fonction ***creer_vec2D*** initialise notre vecteur avec toutes ses valeurs à 0 (zéro).

La fonction ***get*** sert à récupérer une tuile sur la grille aux coordonnées du plateau [INT_MIN, INT_MAX], elle renvoie la tuile si elle est trouvée, et renvoie *NULL* sinon.

La fonction ***set*** pose la tuile aux coordonnées passées en argument.

Joueurs :

Vient ensuite, la représentation de nos joueurs, ici encore, une réflexion rapide nous à apporter toutes les informations que nous **voulions** dans notre structure, ce qui nous a donné le code suivant :

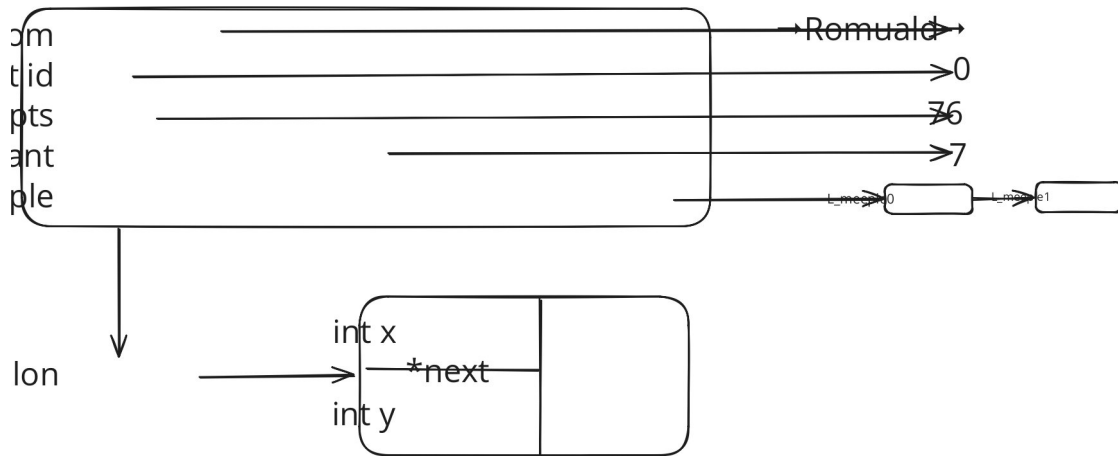
```
typedef struct _Joueur {
    char *nom;
    int id;
    int pts;
    int nb_meeple_restant;
    L_meeple localisation_meeple;
    Color couleur;
} Joueur;
```

La structure *L_meeple* (localisation meeples) enregistre tous les meeples placés par le joueur sur la grille ainsi que leur position.

Liste exhaustive des fonctions :

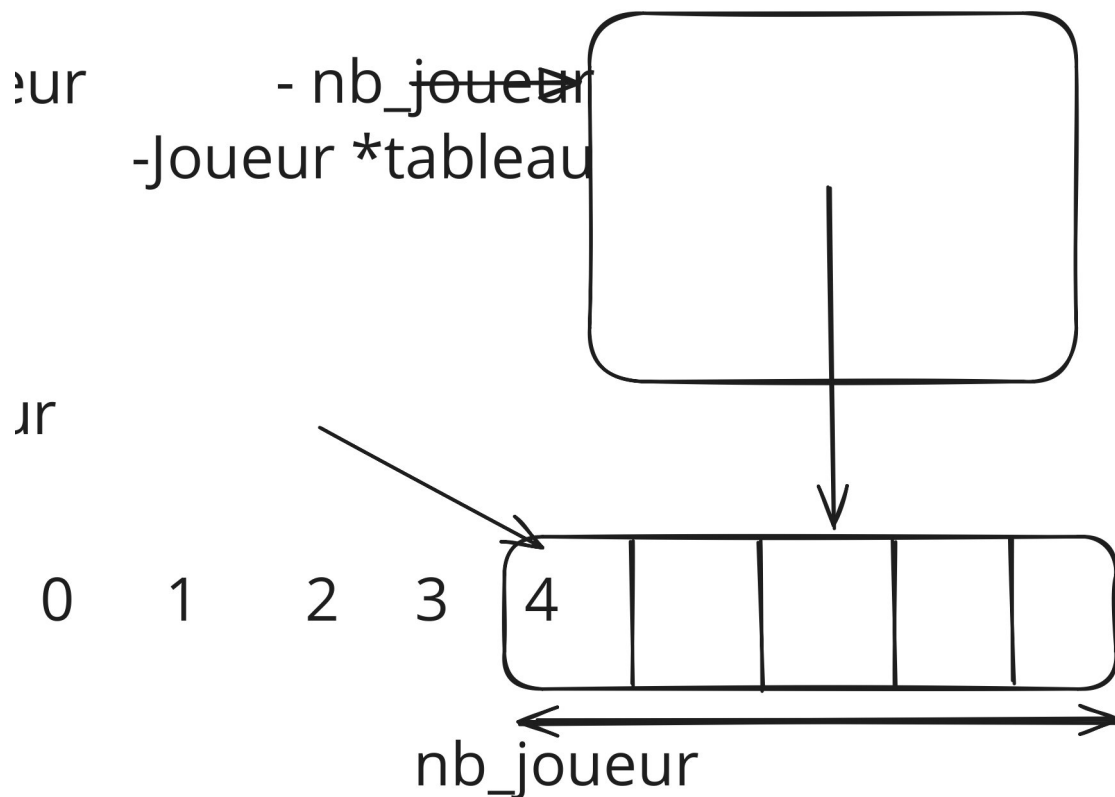
- ***Joueur creer_joueur(int id, int nb_meeple)***

- *void detruire_joueur(Joueur joueur)*
- *ListeJoueurs creer_listejoueurs(int nb_joueurs, int nb_meeple)*
- *void detruire_listejoueurs(ListeJoueurs joueurs)*



Il nous fallait un endroit où réunir tous nos joueurs, donc nous avons créé un tableau où se trouvent tous nos joueurs :

```
typedef struct _ListeJoueurs {
    int nb_joueurs;
    Joueur *tableau;
} ListeJoueurs;
```



Algorithmique :

Recherche :

Cet algorithme s'exécute lorsqu'une procédure de recherche est engagée. Exemple : Lors d'un placement de tuile. Cette fonction permet de renvoyer les points que nous allons pouvoir attribuer par la suite, la recherche effectue également l'analyse du nombre de meeples présents dans un tableau indexé par l'id des joueurs sur la zone recherchée et sauvegarde leurs localisations dans *loc_meeples_all*, une liste chaînée contenant tous les meeples trouvés. La recherche s'arrête quand on arrive sur une tuile déjà vérifiée ou une tuile vide.

Cette fonction est récursive, elle sera appelée sur chaque case de notre recherche.

Il y a un booléen présent dans la recherche nommé *complete*, défini sur *true*, il peut changer de valeur dans le cas où recherche renverrait -1 dans la valeur *tmp*.

Si la zone est complétée, recherche renvoie le nombre de points. Sinon, elle renvoie -1.

La recherche ne revenant pas sur ses pas, on utilise la fonction `amorce_recherche` pour lancer la recherche vers une direction donnée sur une certaine tuile de départ :

Jeu :

Liste des fonctions :

- *Jeu creer_jeu(int nb_joueurs, int nb_meeple, int nb_tuiles)*
- *void detruire_jeu(Jeu jeu)*
- *void attribution_points(Jeu *jeu, L_meeple loc_meeple, int *nb_meeples, int pts)*
- *bool tour(Jeu *jeu, Tuile tuile, int x, int y, int id_meeple, enum Direction position_meeple)*
- *void fin(Jeu *jeu)*

Précisions sur les fonctions :

La fonction ***attribution_points*** n'est appelée qu'après une recherche. Elle va aussi attribuer les points au joueur ayant le plus de meeples.

La fonction ***tour*** gère toutes les actions réalisées par un joueur durant un tour, pour être plus précis : placement de la tuile, du meeples éventuel, de la recherche, et de l'attribution des points. Tour vérifie aussi la présence d'abbaye à proximité et va attribuer les points si nécessaire.

La fonction ***fin*** aura un comportement similaire, mais celle-ci va lancer une recherche à la position de chaque meeples, attribue les points et annonce le gagnant de la partie.

Fichiers :

CSV :

La priorité était d'implémenter une façon de lire un fichier .csv. Nous partons du principe que le fichier est correct. Une seule information importante est à noter : la dernière valeur de chaque ligne du .csv représente la valeur de son milieu. Nous pouvons l'analyser facilement grâce aux valeurs villages et abbaye qui n'apparaissent

qu'au milieu.

Avec la commande `cat tuiles.csv | grep abbaye && cat tuiles.csv | grep village`

```
pre,pre,pre,pre,abbaye
pre,pre,pre,route,abbaye
pre,pre,pre,pre,abbaye
pre,pre,pre,pre,abbaye
pre,pre,pre,pre,abbaye
pre,pre,pre,route,abbaye
pre,route,route,route,village
route,route,pre,route,village
route,route,ville,route,village
route,route,pre,route,village
pre,route,route,route,village
route,route,ville,route,village
route,route,ville,route,village
route,route,route,route,village
```

Ainsi nous aurons trois fonctions principales :

- `int compter_lignes(FILE *f)`
- `bool lire_zone(enum Zone *p, FILE *f)`
- `Pile lire_tuiles_csv(char* nom_fichier)`

Précisions sur certaines fonctions :

compter_lignes nous sert à compter le nombre de lignes (et ainsi de tuiles) présentent dans le fichier .csv. Elle nous sert pour savoir combien de tuiles seront lues par ***lire_tuiles_csv***.

lire_zone va lire une zone (une valeur du csv) et l'ajouter à une zone de la tuile de façon indépendante.

lire_tuile renvoie une pile qui sera remplie par les tuiles lues dans le fichier .csv passé en argument.

Notes : Il faut bien faire attention à récupérer la valeur de fread pour éviter une erreur du compilateur avec l'option -Wall.

Sauvegarder et charger :

Une des fonctionnalités importantes que nous souhaitons importer était la possibilité pour le joueur de sauvegarder et charger des parties. Notre choix a été de sauvegarder les parties sous format binaire pour diminuer leur taille.

La première question à laquelle nous devons répondre était, "Que doit-on enregistrer ?". Après une analyse des besoins pour reprendre une partie dans les meilleures conditions. Voici ce dont nous avons besoin pour d'enregistrer lors d'une sauvegarde :

- L'état de la pile.
- L'état des joueurs
- L'état du plateau avec les tuiles.

Pour répondre à ses besoins, plusieurs fonctions ont été introduites :

- ***void sauvegarder_grille(Vec2D g, FILE *f)***
- ***void sauvegarder_pile(Pile p, FILE *f)***
- ***void sauvegarder_liste_joueurs(ListeJoueurs tab, FILE *f)***
- ***void sauvegarder_joueur(Joueur joueur, FILE *f)***

Ainsi qu'une fonction qui les encapsule toutes dans l'ordre ci-dessus : ***bool sauvegarder_partie(Jeu partie, char *fname)*** pour sauvegarder une partie complète.

Précisions sur certaines fonctions :

La fonction ***sauvegarder_partie*** prend le nom d'un fichier via la variable *fname* et ouvre le fichier (via un pointeur) qui sera ensuite passé à chaque sous-fonctions. Elle précise un numéro de version (devant être changé à chaque modification d'une fonction "sauvegarder").

Conseil : La commande strings sur un fichier binaire de partie permet de savoir facilement la version dudit fichier (binaire) que vous ciblez. En cas de doute, vous pourrez ainsi vous en assurer visuellement, mais la fonction *charger_partie* s'occupe déjà de vérifier la version.

De pair, nous retrouvons les mêmes fonctions pour charger une partie :

- ***Vec2D charger_grille(FILE *f)***
- ***Pile charger_pile(FILE *f)***
- ***ListeJoueurs charger_liste_joueurs(FILE *f)***
- ***Joueur charger_joueur(FILE *f)***

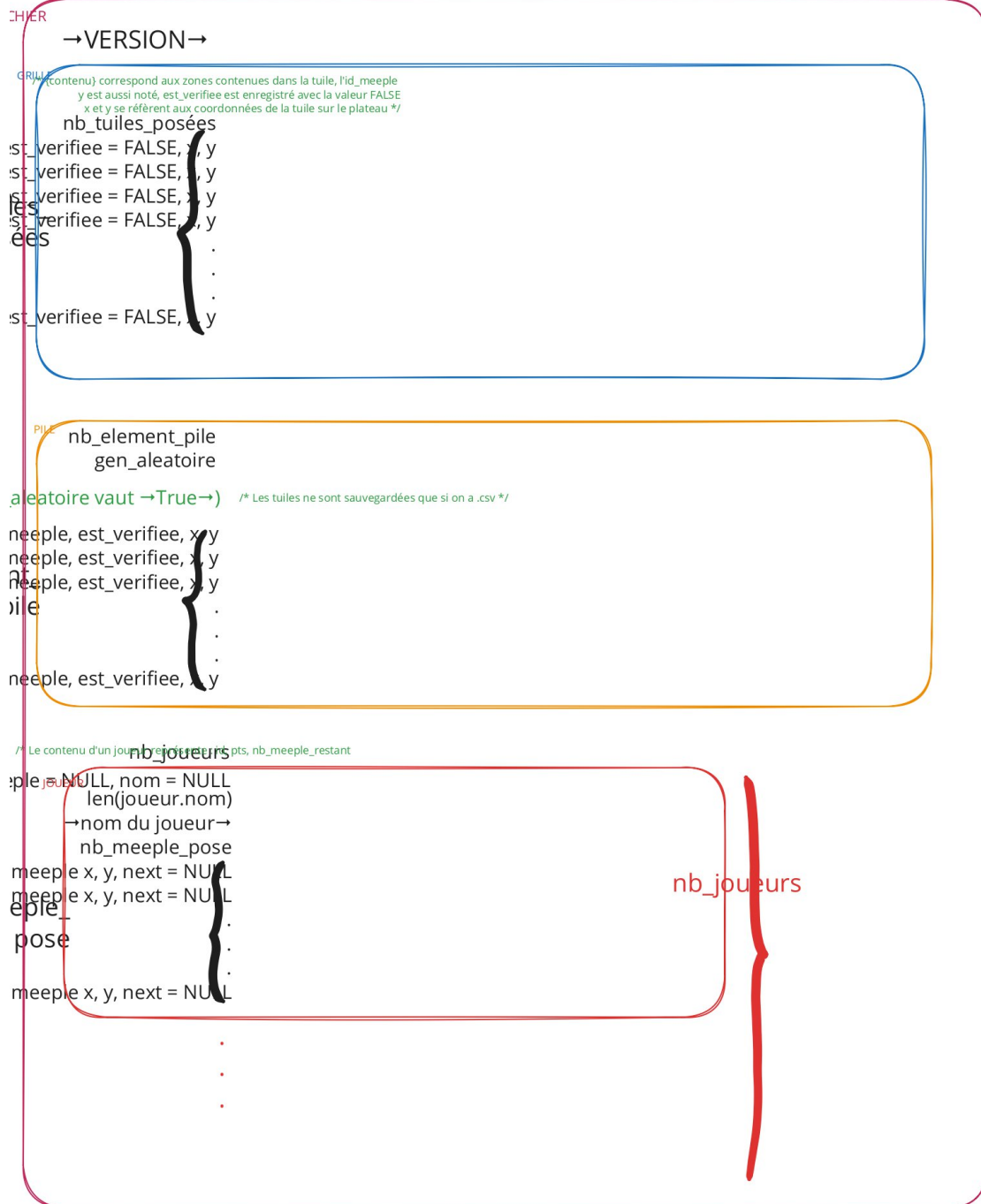
Une fonction encapsulante existe aussi pour charger la partie, toujours dans l'ordre ci-dessus : ***bool charger_partie(Jeu *partie, char *fname)***.

Précisions sur certaines fonctions :

La fonction ***charger_partie*** prend elle aussi le nom d'un fichier via une variable *fname* ouvrira le fichier et passera un pointeur aux autres fonctions. Elle vérifie le numéro de version et poursuit sur le chargement individuel de chaque structure. Si un fichier est incorrecte, le programme sera arrêté.

Ci-dessous, peut être retrouvé un schéma représentant l'agencement des données dans un fichier complet :

er :



Ce schéma est simplifié pour la vision humaine, souvenez-vous que ce ne sont que des bits qui y sont normalement écrits, l'ordre choisi pour représenter le sein de chaque donnée est arbitraire. L'ordre des structures lui respecte l'implémentation actuelle. Des commentaires pertinents sont aussi annotés sur le schéma, tenez en rigueur en cas de questions.

Implémentation graphique :

Nous utilisons la librairie graphique [Raylib](#) pour créer et gérer toute l'interface graphique, la librairie est distribué sous forme pre-compilé et lié statiquement pour permettre l'exécution sans modification de l'environnement de l'utilisateur.

Nous nous servons du `main.c` pour naviguer entre différentes pages, à l'exécution, `main()` configure l'état global tel que les flags OpenGL, la graine aléatoire, et la case mémoire de la structure `Jeu`, puis switch entre chaque page jusqu'à recevoir `P_QUITTER`. Ces pages sont assignées à différentes fonctions qui gèrent chacune l'affichage d'un écran du jeu indépendamment des autres tel que défini dans le rapport d'analyse (page de d'écran titre, page de configuration, page du jeu principal..) et renvoie la prochaine page à afficher tel que définie par l'enum `Page` : Voici les différentes pages :

```
enum Page {  
    P_TITRE,  
    P_JEU,  
    P_CONF_NORMAL, // ces trois enum mènent à la meme page  
    P_CONF_CUSTOM, // avec des options differentes  
    P_CONF_CSV,    // selon le niveau de personalisation requis.  
    P_JOUEURS,  
    P_CHARGER,  
    P_AIDE,  
    P_QUITTER,  
};
```

Chaque page est gérée indépendamment mais certaines fonctionnalités restent récurrentes. C'est pour cela que nous les gérons comme des "widgets" (éléments interactifs) généraux, qui possèdent leurs propres fichiers. Par exemple, les boutons sont considérés comme tels et peuvent être retrouvés dans `bouton.h`, voici une liste exhaustive des widgets existants sur le projet :

- ***bouton.h***
- ***texte.h***
- ***champsaisie.h***
- ***scrollbar.h***
- ***sidebar.h***
- ***cartejoueur.h***

Par exemple, la page de jeu qui est celle où sont posées les tuiles et où la partie se déroule dans son ensemble se sert de la sidebar, de la scrollbar, du champs de saisie etc, la page de configuration inclut des champs de saisies, une scrollbar, casiment toutes les pages font usage de boutons et de texte.

Les champs de saisies sont gérées via `VarString`, une implémentation de chaîne de texte à taille variable qui permet toutes les manipulations textuelles d'une entrée utilisateur simple (ajout, retrait, suppression total, duplication pour l'extraction de l'entrée).

Affichage du model

Une problématique intéressante était de minimiser la poids calculatoire de chaque génération de frame, et laisser le gros du travail au contrôleur. Il a donc été important pour nous de chercher des moyens de conserver des formats de données très rapidement affichable par la vue, et non pas aller chercher de nouveau ces données sur le modèle pour chaque frame. Plusieurs techniques entrent donc en jeu, décrites ci-dessous :

Incluses dans `render.h` sont les fonctions nous servant à allouer des chunks, un chunk est une texture utilisé pour stocker le résultat visuel du placement de plusieurs textures de tuiles, formant une unique bitmap pour tout un jeu de tuile, jusqu'à 32^2 tuiles dans l'implémentation actuelle, même si cette valeur reste configurable dans le header. Le groupement par chunk pré-générés stockés dans la VRAM GPU des sprites de tuiles placés permet un dessin très rapide du plateau tout entier, seulement 4 ou 5 appels `openGL` pour une partie moyenne au lieu d'un par tuile avec une implémentation naïve.

Les textures des tuiles (sprites) sont générées aléatoirement via un algorithme se trouvant dans `render.h`, de façon à respecter la tuile générée aléatoirement, ou celle du `.csv`. Ces textures sont composés à partir de plusieurs 'sous-sprites' représentant les éléments possibles d'une tuile (route, abbaye, différentes formes de villes...) stockés dans une spritesheet pour n'avoir à charger qu'un seul fichier à l'initialisation.

La position des meeples est également 'cachée' à partir des listes chaînées des joueurs, pour ne pas avoir à parcourir la chaîne à chaque frame, on conserve la couleur et les positions $\{x, y\}$ relatives au plateau de chaque meeples posé pour pouvoir rapidement toutes les dessiner.

Il vient alors la question de quand parcourir les listes, et quand se reposer sur les valeurs cachées ? De même pour les chunks et les informations présentes dans la sidebar.

Remarquons que le modèle n'est modifiée qu'à chaque tour, et reste inchangé tant que la vue n'a pas appelé `tour()`, on peut donc rafraichir nos informations sur le modèle qu'après un placement réussi par le joueur, et se baser sur des formats de données plus rapide d'accès pour le dessin habituelle de chaque frame, 60 fois par seconde.